EXVUL

SMART CONTRACT AUDIT REPORT

GEN Smart Contract

OCTOBER 2026

Contents

1. EXECUTIVE SUMMARY **4**
1.1 Methodology 4

2. FINDINGS OVERVIEW **8**
2.1 Project Info And Contract Address 8
2.2 Summary 9
2.3 Key Findings 10

3. DETAILED DESCRIPTION OF FINDINGS **11**
3.1 NFT mint callback overallocates dividends 11
3.2 Public migration rewinds the annual emission state 13
3.3 Public migration restores obsolete deposit caps 15
3.4 Stale GEN TWAP lets a staker evade the LP return cap 17
3.5 A sandwich reduces LP principal without reducing stake power 20
3.6 Missed annual supplements become permanently unrollable 24
3.7 NFT transfers can erase fractional dividend debt 26

4. CONCLUSION **28**

5. APPENDIX **29**
5.1 Basic Coding Assessment 29
5.1.1 Apply Verification Control 29
5.1.2 Authorization Access Control 29
5.1.3 Forged Transfer Vulnerability 29
5.1.4 Transaction Rollback Attack 30
5.1.5 Transaction Block Stuffing Attack 30
5.1.6 Soft Fail Attack Assessment 30
5.1.7 Hard Fail Attack Assessment 31
5.1.8 Abnormal Memo Assessment 31
5.1.9 Abnormal Resource Consumption 31
5.1.10 Random Number Security 32
5.2 Advanced Code Scrutiny 32
5.2.1 Cryptography Security 32
5.2.2 Account Permission Control 32
5.2.3 Malicious Code Behavior 33
5.2.4 Sensitive Information Disclosure 33
5.2.5 System API 33

6. DISCLAIMER **34**

7. REFERENCES **35**

8. About Exvul Security **36**

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **GEN** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

This is a source-package assessment and remediation retest, not a deployed-bytecode or live-economic validation. The seven listed items retain the severities in the canonical issues list, but these ratings are provisional where deployment state or attacker net profit is unknown. Four findings are recorded as FIXED, including M-04 at the project team's direction, while three remain ACKNOWLEDGED, including one partially remediated oracle finding. The M-04 disposition is not supported by an additional code change or a new positive-net-profit proof in the supplied package. The template's PASSED conclusion records completion of the review only; it does not certify a safe deployment or state that all findings were independently verified as fixed.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

IMPACT

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
GEN	13/10/2026 - 19/10/2026	Solidity

Repository
Local remediation source package; repository URL not provided

Commit Hash
89d94d2 (package-declared short hash; Git ancestry unverified)

The assessed production scope is six GEN Solidity contracts. Project interfaces and third-party-derived math libraries were used as context, not counted as separately audited implementations. The remediation package identifies a BSC/BEP-20 deployment target and declares short commit 89d94d2, but contains no Git metadata, repository URL, deployed addresses, implementation-bytecode match, or live-state snapshot. The canonical source issues originated from Lark document revision 26.

Patched-package verification commands were `forge build` with Solidity 0.8.28 and optimizer runs 200, followed by `cd verification-fixed && forge test -vv`. The build passed and the targeted remediation suite observed 7 passed, 0 failed. The tests cover callback dividend accounting, removed migration selectors, pre-redemption oracle refresh, oracle-reconfiguration stale-cache carryover, end-of-schedule roll behavior, NFT dividend rounding, and NFT V3 minimum output. They do not establish positive net profit or current on-chain exploitability. The supplied deployment scripts were excluded and are not self-contained in this package because referenced `_shared` modules are absent.

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	1
MEDIUM	4
LOW	2
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
HIGH	NFT mint callback overallocates dividends	Fixed
MEDIUM	Public migration rewinds the annual emission state	Fixed
MEDIUM	Public migration restores obsolete deposit caps	Fixed
MEDIUM	Stale GEN TWAP lets a staker evade the LP return cap	Acknowledge
MEDIUM	A sandwich reduces LP principal without reducing stake power	Fixed
LOW	Missed annual supplements become permanently unrollable	Acknowledge
LOW	NFT transfers can erase fractional dividend debt	Acknowledge

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 NFT mint callback overallocates dividends

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/NFTMining.sol`

DESCRIPTION:

The purchase flow calls `_safeMint` before it records `mintTime` and increments `totalMinted`. The ERC721 receiver callback therefore runs after the new token exists but before the dividend denominator includes it. A whitelisted receiver can deposit GEN into the vault and transfer the new NFT from that callback. The transfer path then synchronizes the deposit against the stale denominator and overallocates the callback participant.

```
        _syncDividend();

        uint256 A;
        uint256 paid;
        if (payToken == 0) {
            paid = priceUsd;
            IERC20(usdt).safeTransferFrom(msg.sender, address(this), paid);
            uint256 aBefore = IERC20(antfun).balanceOf(address(this));
            _swap(usdt, antfun, paid);
            A = IERC20(antfun).balanceOf(address(this)) - aBefore;
        } else if (payToken == 1) {
            uint256 px = IGenTokenViews(genToken).antfunUsd();
            if (px == 0) revert OracleNotReady();
            paid = FullMath.mulDiv(priceUsd, 1e18, px);
            IERC20(antfun).safeTransferFrom(msg.sender, address(this), paid);
            A = paid;
        } else {
            revert InvalidPayToken(payToken);
        }

        (uint256 gBurned, uint256 gForLp, uint256 lpMinted) = _distributeProceeds(A);

        tokenId = ++_nextId;
        hasMinted[msg.sender] = true;
        _safeMint(msg.sender, tokenId);
        mintTime[tokenId] = uint64(block.timestamp);
        totalMinted += 1;
```

```
        emit Purchased(tokenId, msg.sender, payToken, paid, A, gBurned, gForLp, lpMinted);
```

```
    function _update(address to, uint256 tokenId, address auth) internal override returns (
    address) {
        address from = _ownerOf(tokenId);
        _syncDividend();

        if (from != address(0)) _settleDividend(from);
        if (to != address(0) && to != from) _settleDividend(to);

        address prev = super._update(to, tokenId, auth);

        if (from != address(0)) divDebt[from] = FullMath.mulDiv(balanceOf(from), accDivPerNFT
    , ACC_PRECISION);
        if (to != address(0) && to != from) divDebt[to] = FullMath.mulDiv(balanceOf(to),
    accDivPerNFT, ACC_PRECISION);
```

IMPACT:

A malicious whitelisted buyer can take GEN that belongs to existing NFT holders. The loss scales with the GEN introduced during the callback. The reviewed package does not establish the live whitelist or deployed balances.

Verification boundary: The patched package synchronizes newly accrued dividends and finalizes `mintTime` and `totalMinted` before `_safeMint`. A callback retest confirmed that `totalMinted` is already two in the receiver callback, the callback participant can claim only its funded share, and the prior holder remains fully payable. The new NFT storage variables consume one former gap slot without shifting existing state.

RECOMMENDATIONS:

Synchronize existing dividends and finalize denominator accounting before the receiver callback. Keep `nonReentrant` protection and test a receiver that funds the vault and transfers the token from `onERC721Received`.

```
         tokenId = ++_nextId;
         hasMinted[msg.sender] = true;
+        _syncDividend();
+        mintTime[tokenId] = uint64(block.timestamp);
+        totalMinted += 1;
         _safeMint(msg.sender, tokenId);
-        mintTime[tokenId] = uint64(block.timestamp);
-        totalMinted += 1;
```

3.2 Public migration rewinds the annual emission state

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/LpMining.sol`

DESCRIPTION:

`initializeEmission` is an unrestricted versioned reinitializer. If version 3 has not already been consumed, any account can call it after annual rolls have advanced and reset `lastRolledYear` to one. The caller can then replay expired years. Each replay uses the current SecondYearPool balance while overwriting the stored daily supplement for a historical year.

```
    function _setDefaultQuotaCaps() private {
        perAccountCap30 = 100e18;
        perAccountCap60 = 1_000e18;
        perAccountCapRest = 10_000e18;
        networkCap5 = 200_000e18;
        networkCap30 = 100_000e18;
        networkCap60 = 200_000e18;
    }

    function initializeQuotaCaps() external reinitializer(2) {
        _setDefaultQuotaCaps();
    }

    function initializeEmission() external reinitializer(3) {
        lastRolledYear = 1;
    }
```

```
    function rollYear() external {
        require(secondYearPool != address(0), "no secondYearPool");
        uint256 lrY = lastRolledYear == 0 ? 1 : lastRolledYear;
        uint256 target = lrY + 1;
        require(target >= 2 && target <= 10, "no roll");
        require(currentYear() >= target, "year not started");
        uint256 accrued = ISecondYearPool(secondYearPool).releaseAllToMining();
        yearSupplementPerDay[target] = accrued / yearDays;
        lastRolledYear = target;
```

```
        emit YearRolled(target, accrued, yearSupplementPerDay[target]);
```

IMPACT:

An unauthorized caller can corrupt annual emission accounting, move current GEN into an expired schedule bucket, and erase a previously funded supplement rate. Exploitability on a deployment is conditional on reinitializer version 3 remaining unused; deployment state was not included in the review package.

Verification boundary: The patched implementation removes `initializeEmission()` from both source and compiled ABI, while the primary initializer still sets `lastRolledYear` to one. A raw call to the removed selector through the proxy reverted and left the emission cursor unchanged. Existing proxies remain exposed until the patched implementation is installed, and the patch does not repair state already corrupted before remediation.

RECOMMENDATIONS:

Restrict the migration to the configured authority and make it preserve an already initialized roll cursor. Execute and consume migrations atomically with the proxy upgrade, then verify initializer version and emission state on-chain.

```
-     function initializeEmission() external reinitializer(3) {
-         lastRolledYear = 1;
+     function initializeEmission() external restricted reinitializer(3) {
+         if (lastRolledYear == 0) lastRolledYear = 1;
      }
```

3.3 Public migration restores obsolete deposit caps

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/LpMining.sol`

DESCRIPTION:

`initializeQuotaCaps` is an unrestricted versioned reinitializer. If version 2 has not already been consumed, any account can call it and execute `_setDefaultQuotaCaps()`. This overwrites quota caps that the authority may previously have reduced in response to risk, liquidity, or business constraints.

```
    function _setDefaultQuotaCaps() private {
        perAccountCap30 = 100e18;
        perAccountCap60 = 1_000e18;
        perAccountCapRest = 10_000e18;
        networkCap5 = 200_000e18;
        networkCap30 = 100_000e18;
        networkCap60 = 200_000e18;
    }

    function initializeQuotaCaps() external reinitializer(2) {
        _setDefaultQuotaCaps();
    }
```

IMPACT:

Eligible accounts can deposit above the authority's intended lower limits after an unauthorized reset. The effect is conditional on reinitializer version 2 remaining unused and on current caps differing from the hard-coded defaults.

Verification boundary: The patched implementation removes `initializeQuotaCaps()` from both source and compiled ABI, while the primary initializer still installs all six default caps. A raw call to the removed selector through the proxy reverted and left custom caps unchanged. Existing proxies remain exposed until the patched implementation is installed, and future upgrades must not reintroduce unrestricted version 2 or version 3 migrations.

RECOMMENDATIONS:

Apply the same authority check used by other configuration methods, consume the migration atomically with the intended upgrade, and avoid overwriting nonzero operator-configured caps unless the migration explicitly requires it.

```
-    function initializeQuotaCaps() external reinitializer(2) {
+    function initializeQuotaCaps() external restricted reinitializer(2) {
         _setDefaultQuotaCaps();
     }
```

3.4 Stale GEN TWAP lets a staker evade the LP return cap

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

`contracts/LpMining.sol`

DESCRIPTION:

`redeem` calculates the current LP value and fixes `lpReturn` using the cached GEN TWAP. The function does not require that price to be fresh before applying the return cap. A later pair transfer in the same redemption can refresh the oracle, but that occurs only after the stale value has already determined how much LP is returned.

```
    function fairLpValueUsd(uint256 lpAmount) public view returns (uint256) {
        IPancakePairMinimal pair = IPancakePairMinimal(genAntfunPair);
        (uint112 r0, uint112 r1, ) = pair.getReserves();
        (uint256 rGen, uint256 rAntfun) = genIsToken0
            ? (uint256(r0), uint256(r1))
            : (uint256(r1), uint256(r0));

        uint256 lpSupply = pair.totalSupply();
        if (lpSupply == 0) return 0;

        uint256 pAntfun = IGenTokenViews(genToken).antfunUsd();
        uint256 pGen = FullMath.mulDiv(IGenTokenViews(genToken).genPriceAntfun(),
    pAntfun, 1e18);

        uint256 k = rGen * rAntfun;
        uint256 sqrtK = Math.sqrt(k);
        uint256 sqrtPP = Math.sqrt(pGen * pAntfun);
        uint256 poolValueUsd = 2 * sqrtK * sqrtPP / 1e18;

        return FullMath.mulDiv(poolValueUsd, lpAmount, lpSupply);
```

```
    function redeem(uint256 stakeId) external nonReentrant {
        if (stakeId == 0 || stakeId > nextStakeId) revert InvalidStakeId(stakeId);
        StakeRecord storage record = _stakes[stakeId];
        if (record.owner != msg.sender) revert NotStakeOwner(stakeId, msg.sender);
        if (record.status != STATUS_ACTIVE) revert NotActive(stakeId, record.status
    );
```

```
        uint256 today = currentDay();
        if (today <= record.maturityDay) revert NotMatured(stakeId, record.
    maturityDay, today);

        if (record.lpAmount == 0) revert InvalidStakeState(stakeId);
        uint256 currentLpValue = fairLpValueUsd(record.lpAmount);
        if (currentLpValue == 0) revert OracleNotReady(stakeId);

        uint256 cap = FullMath.mulDiv(record.usdValue, record.capMulBps, BPS);
        uint256 lpReturn = currentLpValue <= cap
            ? record.lpAmount
            : FullMath.mulDiv(record.lpAmount, cap, currentLpValue);
        uint256 excess = record.lpAmount - lpReturn;

        record.status = STATUS_REDEEMED;
        accountHolding[record.owner] -= record.usdValue;

        address lp = genAntfunPair;
        uint256 genBefore = IERC20(genToken).balanceOf(address(this));
        uint256 antBefore = IERC20(antfun).balanceOf(address(this));
        IERC20(lp).forceApprove(router, lpReturn);
        IPancakeRouter02(router).removeLiquidity(
            genToken, antfun, lpReturn, 0, 0, address(this), block.timestamp
```

```
    function pokeGenTwap() public {
        (uint256 cum, uint32 ts) = _currentCum();
        if (checkpointTs == 0) {
            priceCumCheckpoint = cum;
            checkpointTs = ts;
            return;
        }
        uint32 dt;
        unchecked {
            dt = ts - checkpointTs;
        }
        if (dt < genTwapWindow) return;
        uint256 avgUQ;
        unchecked {
            avgUQ = (cum - priceCumCheckpoint) / dt;
        }
        lastGenPriceAntfun = FullMath.mulDiv(avgUQ, 1e18, 1 << 112);
        priceCumCheckpoint = cum;
        checkpointTs = ts;
    }

    function genPriceAntfun() public view returns (uint256) {
```

```
        return lastGenPriceAntfun;
    }
```

IMPACT:

A mature staker can receive LP value above the configured redemption cap when the cached GEN price materially understates the current market price. Practical loss depends on oracle age, price movement, stake size, and pool depth.

Verification boundary: The patch adds `pokeGenTwap()` to the interface and calls it before valuation. A focused test confirmed that a real refresh changes the redemption cap calculation. Remediation is incomplete: `setGenTwap()` resets the checkpoint without clearing `lastGenPriceAntfun`, and the first subsequent poke only seeds the new checkpoint. An actual `GenToken` regression test confirmed that the previous pair's nonzero price remains readable for one configured window. Long-idle refreshes also average over the full elapsed interval rather than a bounded latest window.

RECOMMENDATIONS:

Refresh the GEN oracle before valuation and require the observation to satisfy the configured freshness window. Revert when a safe price cannot be established rather than allowing the transfer side effect to update the oracle after the cap is fixed.

```
     function redeem(uint256 stakeId) external nonReentrant {
+        IGenTokenViews(genToken).pokeGenTwap();
+        require(IGenTokenViews(genToken).isGenTwapFresh(), "stale GEN oracle");
         uint256 currentLpValue = fairLpValueUsd(record.lpAmount);
```

3.5 A sandwich reduces LP principal without reducing stake power

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/LpMining.sol`

DESCRIPTION:

`deposit` performs payment swaps and liquidity creation without an enforceable minimum LP output. The resulting stake records the actual LP amount, but `stakePower` is based on the full nominal payment. A depositor or searcher can sandwich the conversion so the position receives much less LP while retaining reward weight derived from the pre-manipulation input amount.

```
        uint256 usdValue;
        if (payToken == 0) {
            usdValue = payAmount;
        } else {
            usdValue = FullMath.mulDiv(payAmount, IGenTokenViews(genToken).
    antfunUsd(), 1e18);
        }
        if (usdValue == 0) revert ZeroUsdValue();

        _consumeQuota(msg.sender, usdValue);

        address payTokenAddr = payToken == 0 ? usdt : antfun;
        IERC20(payTokenAddr).safeTransferFrom(msg.sender, address(this), payAmount)
    ;

        uint256 lpAmount = _swapAndAddLiquidity(payToken, payAmount);

        uint256 stakePower = FullMath.mulDiv(usdValue, cfg.coefBps, BPS);

        uint256 compensationBps = 50 * (today - 1);
        uint256 basePower = FullMath.mulDiv(stakePower, BPS + compensationBps, BPS)
    ;

        stakeId = ++nextStakeId;
        uint256 maturityDay = today + cfg.lockDays;
        _stakes[stakeId] = StakeRecord({
            id: stakeId,
```

```
            owner: msg.sender,
            payToken: payToken,
            payAmount: payAmount,
            lpAmount: lpAmount,
            usdValue: usdValue,
            lockMonths: lockMonths,
            coefBps: cfg.coefBps,
            stakeDay: today,
            maturityDay: maturityDay,
            capMulBps: cfg.capMulBps,
            stakePower: stakePower,
            status: STATUS_ACTIVE,
            compensationBps: compensationBps,
            basePower: basePower
        });
```

```
    function _swapAndAddLiquidity(uint8 payToken, uint256 payAmount) internal
returns (uint256 lpAmount) {
        uint256 antfunAmount;
        if (payToken == 0) {
            antfunAmount = v3SwapRouter != address(0)
                ? _swapV3(usdt, antfun, payAmount, usdtAntfunV3Fee)
                : _swap(usdt, antfun, payAmount);
        } else {
            antfunAmount = payAmount;
        }

        uint256 halfAntfun = antfunAmount / 2;
        uint256 genAmount = _swap(antfun, genToken, halfAntfun);
        uint256 remainingAntfun = antfunAmount - halfAntfun;

        IERC20(genToken).forceApprove(router, genAmount);
        IERC20(antfun).forceApprove(router, remainingAntfun);
        (uint256 usedGen, uint256 usedAnt, uint256 lpAmount_) = IPancakeRouter02(
router).addLiquidity(
            genToken, antfun, genAmount, remainingAntfun, 0, 0, address(this),
block.timestamp
        );
        lpAmount = lpAmount_;

        if (dustReceiver != address(0)) {
            uint256 dustGen = genAmount - usedGen;
            uint256 dustAnt = remainingAntfun - usedAnt;
            if (dustGen > 0) IERC20(genToken).safeTransfer(dustReceiver, dustGen);
            if (dustAnt > 0) IERC20(antfun).safeTransfer(dustReceiver, dustAnt);
        }
```

```
        IERC20(genToken).forceApprove(router, 0);
        IERC20(antfun).forceApprove(router, 0);
    }

    function _swap(address tokenIn, address tokenOut, uint256 amountIn) internal
    returns (uint256 amountOut) {
        IERC20(tokenIn).forceApprove(router, amountIn);
        address[] memory path = new address[](2);
        path[0] = tokenIn;
        path[1] = tokenOut;
        uint256 before = IERC20(tokenOut).balanceOf(address(this));
        IPancakeRouter02(router).
    swapExactTokensForTokensSupportingFeeOnTransferTokens(
            amountIn, 0, path, address(this), block.timestamp
        );
        amountOut = IERC20(tokenOut).balanceOf(address(this)) - before;
    }

    function _swapV3(address tokenIn, address tokenOut, uint256 amountIn, uint24
    fee) internal returns (uint256 amountOut) {
        IERC20(tokenIn).forceApprove(v3SwapRouter, amountIn);
        uint256 before = IERC20(tokenOut).balanceOf(address(this));
        IPancakeV3SwapRouter(v3SwapRouter).exactInputSingle(
            IPancakeV3SwapRouter.ExactInputSingleParams({
                tokenIn: tokenIn,
                tokenOut: tokenOut,
                fee: fee,
                recipient: address(this),
                deadline: block.timestamp,
                amountIn: amountIn,
                amountOutMinimum: 0,
                sqrtPriceLimitX96: 0
            })
        );
        amountOut = IERC20(tokenOut).balanceOf(address(this)) - before;
        IERC20(tokenIn).forceApprove(v3SwapRouter, 0);
```

IMPACT:

A self-sandwich can lower the economic principal required for a given recorded stake power, while a third-party sandwich can directly reduce a depositor's LP principal. The reward loss depends on the off-chain Merkle allocation policy, which was outside the supplied source scope.

Verification boundary: This finding is marked FIXED at the project team's direction. The supplied patch leaves the relevant `LpMining` deposit, V2/V3 swap, liquidity-minimum, and nominal-input stake-power logic unchanged, and the added NFT V3 route also passes `amountOutMinimum = 0`. No new positive-net self-sandwich or excess-reward proof independently established technical remediation; reward allocation uses an authority-published Merkle root.

RECOMMENDATIONS:

Compute a conservative minimum LP amount from fresh oracle prices, enforce it after the complete swap-and-add-liquidity path, and derive reward weight from the economic value or actual LP received rather than the nominal payment alone.

```
-         uint256 lpAmount = _swapAndAddLiquidity(payToken, payAmount);
+         uint256 minLp = oracleQuotedLp(payToken, payAmount) * 99 / 100;
+         uint256 lpAmount = _swapAndAddLiquidity(payToken, payAmount, minLp);
+         require(lpAmount >= minLp, "insufficient LP");
```

3.6 Missed annual supplements become permanently unrollable

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

`contracts/LpMining.sol`

DESCRIPTION:

After the ten-year schedule ends, `currentYear()` returns zero. If one or more annual rolls were not completed by then, `rollYear` can no longer advance because its year checks can never be satisfied. The SecondYearPool balance remains outside the daily supplement schedule.

```
    function yearOf(uint256 day) public view returns (uint256) {
        if (day <= STARTUP_DAYS) return 0;
        uint256 y = (day - STARTUP_DAYS - 1) / yearDays + 1;
        return y > 10 ? 0 : y;
    }

    function currentYear() public view returns (uint256) {
        return yearOf(currentDay());
    }
```

```
    function rollYear() external {
        require(secondYearPool != address(0), "no secondYearPool");
        uint256 lrY = lastRolledYear == 0 ? 1 : lastRolledYear;
        uint256 target = lrY + 1;
        require(target >= 2 && target <= 10, "no roll");
        require(currentYear() >= target, "year not started");
        uint256 accrued = ISecondYearPool(secondYearPool).releaseAllToMining();
        yearSupplementPerDay[target] = accrued / yearDays;
        lastRolledYear = target;
        emit YearRolled(target, accrued, yearSupplementPerDay[target]);
```

IMPACT:

Delayed operations near or after the schedule boundary can permanently strand queued GEN from annual emission accounting. The issue is time-gated and does not create a present-day theft path,

which limits its severity.

Verification boundary: The patched source leaves `yearOf`, `currentYear`, and `rollYear` unchanged. A remediation regression test confirmed that normal annual rolls still fail after year ten. The pool administrator can still release or burn tokens; the irrecoverability claim is limited to the normal emission schedule.

RECOMMENDATIONS:

Clamp the schedule day to its final supported boundary or introduce an explicit post-schedule settlement path. Add tests that skip multiple years and attempt the final outstanding roll after the ten-year endpoint.

```
     function currentYear() public view returns (uint256) {
-        return yearOf(currentDay());
+        uint256 day = currentDay();
+        uint256 lastDay = STARTUP_DAYS + 10 * yearDays;
+        return yearOf(day > lastDay ? lastDay : day);
     }
```

3.7 NFT transfers can erase fractional dividend debt

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

`contracts/NFTMining.sol`

DESCRIPTION:

Dividend debt is stored per address after rounding an aggregate balance. During a transfer, `_update` recalculates debt separately for the sender and receiver. Splitting one rounded aggregate into several smaller rounded values can discard fractional debt. Repeated small deposits and transfers accumulate claimable units that were never funded in the dividend vault.

```
    uint256 public accDivPerNFT;
    address public dividendVault;
    uint256 public lastVaultBalance;
    mapping(address => uint256) public divDebt;
    mapping(address => uint256) public divPending;
```

```
    function _settleDividend(address a) internal {
        uint256 accrued = FullMath.mulDiv(balanceOf(a), accDivPerNFT, ACC_PRECISION
    );
        uint256 owed = accrued - divDebt[a];
        if (owed > 0) divPending[a] += owed;
        divDebt[a] = accrued;
    }

    function claimableDividend(address a) external view returns (uint256) {
        uint256 accrued = FullMath.mulDiv(balanceOf(a), accDivPerNFT, ACC_PRECISION
    );
        return divPending[a] + (accrued - divDebt[a]);
```

```
    function _update(address to, uint256 tokenId, address auth) internal override
    returns (address) {
        address from = _ownerOf(tokenId);
        _syncDividend();

        if (from != address(0)) _settleDividend(from);
        if (to != address(0) && to != from) _settleDividend(to);
```

```
        address prev = super._update(to, tokenId, auth);

        if (from != address(0)) divDebt[from] = FullMath.mulDiv(balanceOf(from),
    accDivPerNFT, ACC_PRECISION);
        if (to != address(0) && to != from) divDebt[to] = FullMath.mulDiv(balanceOf
    (to), accDivPerNFT, ACC_PRECISION);
```

IMPACT:

Attackers can create an accounting deficit and eventually cause later dividend claims to fail or consume GEN intended for other holders. The demonstrated discrepancy arises at the smallest token-unit scale per rounding event, so practical impact depends on repetition cost and token precision.

Verification boundary: The per-address rounded dividend-debt logic is unchanged. A remediation regression test reproduced the one-smallest-unit overclaim and the later unpayable claim. Material extraction after gas and repeated transfers was not demonstrated.

RECOMMENDATIONS:

Track debt at token-level precision or carry fractional remainders across transfers so that moving NFTs cannot lower aggregate debt. Add invariant tests asserting that total claimable dividends never exceed funded and unclaimed vault assets.

```
-     mapping(address => uint256) public divDebt;
+     mapping(uint256 => uint256) public divDebtPerToken;
```

4. CONCLUSION

In this audit, we thoroughly analyzed **GEN** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Findings in Sections 3.2 and 3.3
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Findings in Sections 3.2 and 3.3
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul